

JAVA | POO | UML | TESTES

Blackjack em Camadas

Roadmap guiado para concluir o projeto atual
sem apagar o trabalho intelectual do estudante.

Diagnóstico factual, contratos, exercícios, testes e critérios de pronto
Snapshot do projeto: 24 de agosto de 2026



ANTES DE COMEÇAR

O propósito deste roteiro

Este documento transforma o estado atual do seu projeto em um caminho de estudo. Ele não substitui seu raciocínio com classes prontas: mostra fatos, dependências, contratos, perguntas e testes que permitem construir cada parte conscientemente.

RESULTADO DA AUDITORIA

O projeto compila e os três testes atuais passam. Entretanto, ele ainda não executa uma rodada: cria os objetos, mostra a mensagem de preparação e retorna ao menu. A prioridade imediata é fazer a mão conservar cartas.

Como este PDF deve ser usado

1. Leia uma fase inteira antes de alterar o código.
2. Escreva os testes-alvo ou, no mínimo, descreva Dado/Quando/Então.
3. Implemente o menor comportamento que satisfaz o contrato.
4. Execute `mvn clean test`.
5. Explique em voz alta por que o método pertence àquela classe.
6. Atualize o PUMML apenas depois que o código representar a relação.

O que este material não fará

- Não implementará apostas, split, double down, seguro ou múltiplos jogadores no primeiro ciclo.
- Não recomendará getters e setters automáticos para todos os campos.
- Não tratará o terminal como lugar das regras do domínio.
- Não exigirá conceitos que ainda não melhoram o projeto, como herança prematura.
- Não confundirá código que compila com comportamento correto.

REGRA DE ESTUDO

Cada mudança precisa responder a três perguntas: qual problema ela resolve, qual objeto possui os dados necessários e qual teste prova o comportamento.

LEGENDA

Como ler diagnósticos e exercícios

Rótulo	Significado	Sua ação
ESTÁVEL E TESTADO	Existe, representa a intenção atual e possui ao menos uma verificação útil.	Preserve; mude somente com teste.
PARCIAL	Há uma base correta, mas faltam invariantes ou comportamento.	Complete em incrementos pequenos.
AUSENTE	A responsabilidade ainda não foi implementada.	Defina contrato antes do corpo.
DIVERGENTE	Código, PUML ou documentação contam histórias diferentes.	Escolha a fonte de verdade e sincronize.

Símbolos editoriais

CONCEITO CENTRAL

Explica a ideia que deve sobreviver ao exercício.

PARE E PENSE

Interrompa a leitura e formule sua própria resposta.

ARMADILHA

Mostra um caminho que compila, mas enfraquece o modelo.

DIAGNÓSTICO

Relaciona um sintoma atual com sua causa concreta.

NAVEGAÇÃO

Sumário

O propósito deste roteiro 2

 Como este PDF deve ser usado 2

 O que este material não fará 2

Como ler diagnósticos e exercícios 3

 Símbolos editoriais 3

Snapshot técnico do projeto 8

 Execução observada 8

 Leitura correta desse resultado 8

 Matriz de saúde por componente 9

 Grafo de objetos atual 10

 Débito de inicialização em Blackjack 10

Escopo do Blackjack mínimo 11

 MVP recomendado 11

 Explicitamente fora do primeiro ciclo 11

 Vocabulário de regras 12

 Tabela de decisões do projeto 12

Mapa de dependências 13

 Ordem recomendada 13

 Por que não começar pelo menu? 14

Ler o sistema antes de alterá-lo 15

Fazer a mão e os participantes conservarem estado 18

Fechar o domínio de Carta 21

Completar o ciclo de vida do Baralho 24

A transferência de uma carta	27
Invariante de conservação	27
Experimento de referência	27
Transformar Blackjack em coordenador	28
Calcular pontuação no lugar correto	31
Teste manual de múltiplos ases	34
Ficha de previsão	34
Modelar turnos e resultado da rodada	35
Máquina de estados mínima	38
Tabela de transições para completar	38
Conectar o domínio à interface	39
Usar testes, UML e README como especificação	42
Matriz completa de testes	45
Modelo Dado / Quando / Então	46
Arrange, Act, Assert	46
Aleatoriedade sem testes frágeis	46
Testes que parecem bons, mas não bastam	47
Ordem de execução recomendada	47
Laboratório de UML	48
Problemas atuais de blackjack.puml	48
Modelo-alvo conceitual	49
Relações e multiplicidades	50
Esqueleto PlantUML para você completar	50
Checklist de sincronização do diagrama	51
Perguntas de revisão	51

Contratos do modelo-alvo	52
Invariantes por classe	53
Sinais de encapsulamento saudável	53
Definição de pronto do MVP	54
Domínio e estado	54
Fluxo da partida	54
Qualidade técnica	55
Documentação e modelo	55
Roteiro de aceitação manual	56
Registro de evidência	56
Plano de estudo em doze sessões	57
Ritual de cada sessão	57
Ficha reutilizável de implementação	58
Pistas graduais	59
1. Mao não conserva cartas	59
2. Jogador perde a mão	59
3. Carta aceita XPTO	60
4. Comprar não reduz o baralho	60
5. Distribuição cria cartas novas	61
6. A + A soma 22	61
7. Teste de shuffle falha às vezes	62
8. Nome completo quebra o menu	62
Glossário aplicado	63
Sua próxima ação concreta	65
Sequência das próximas três entregas	65

Perguntas finais de professor 65

CAPÍTULO 01

01

Snapshot técnico do projeto

Uma fotografia factual do que existe em 24 de agosto de 2026, antes que novas mudanças alterem o diagnóstico.

BASELINE CONFIRMADO

`mvn clean test` recompila 8 fontes, executa 3 testes e termina com **BUILD SUCCESS**. Isso prova que a base compila e que três comportamentos específicos passam; não prova que o Blackjack esteja jogável.

Execução observada

TRAÇO DE EXECUÇÃO

- 1) O menu é exibido.
- 2) A opção 1 solicita um nome.
- 3) App cria Jogador, Baralho, Dealer e Blackjack.
- 4) A mensagem "Estamos preparando seu jogo..." aparece.
- 5) Nenhuma carta é distribuída.
- 6) O programa volta ao menu.

Leitura correta desse resultado

- A infraestrutura Maven e JUnit está funcionando.
- Naipes, Carta e a montagem inicial do Baralho já têm uma base útil.
- O sistema ainda não possui fluxo de partida; instanciar Blackjack não executa automaticamente uma rodada.
- O objeto local chamado jogo é criado em App e não recebe nenhuma mensagem depois disso.

ERRO DE INTERPRETAÇÃO

Objeto construído não significa comportamento executado. Um construtor estabelece um estado inicial; uma operação explícita deve iniciar ou conduzir a rodada.

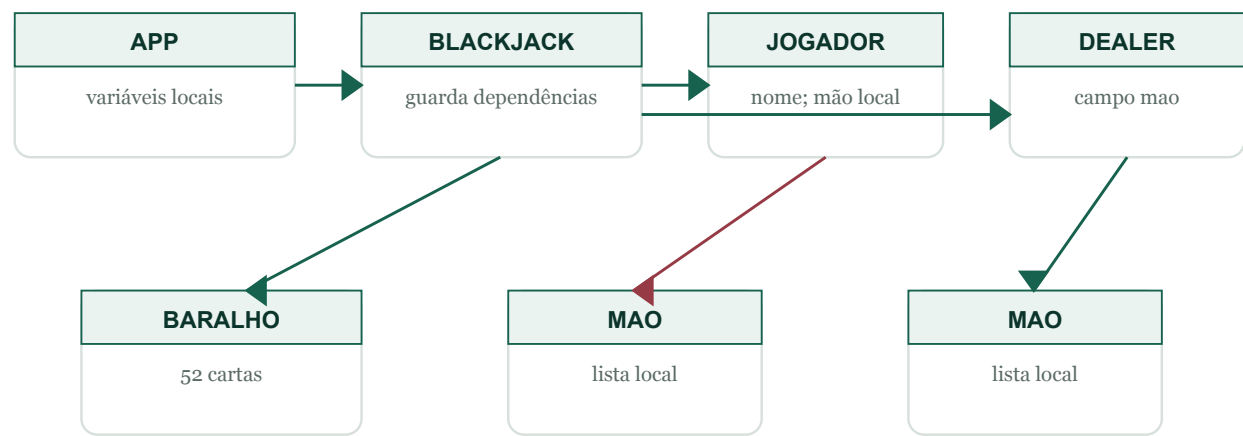
Matriz de saúde por componente

Componente	Estado	Base existente	Próximo avanço
Naípe	ESTÁVEL E TESTADO	Enum com quatro valores e símbolo.	Aparecer no PUML; ampliar testes apenas quando útil.
Carta	PARCIAL	Campos finais, validação básica, normalização e getters.	Impedir ranks inválidos; definir valor-base, texto e igualdade.
Baralho	PARCIAL	Monta 52 cartas e informa quantidade.	Embaralhar, comprar, vazio, conteúdo verificável e testes de unicidade.
Mao	AUSENTE	A classe existe, mas a lista é local do construtor.	Criar campo persistente e operações de domínio.
Jogador	PARCIAL	Conserva nome.	Encapsular nome e conservar uma mão própria.
Dealer	PARCIAL	Conserva uma Mao como campo.	Encapsular, tornar final e definir política de compra.
Blackjack	AUSENTE	Recebe dependências no construtor.	Remover objetos redundantes e coordenar distribuição, turnos e resultado.
App	PARCIAL	Menu e criação inicial funcionam.	Entrada robusta, usar o jogo e exibir uma rodada.
Testes	PARCIAL	Três testes reais passam.	Separar por classe e cobrir invariantes e regras.
PUML	DIVERGENTE	Esboça as relações principais.	Corrigir sintaxe e sincronizar com código atual/alvo.
README	PARCIAL	Comandos Maven estão corretos.	Documentar escopo, regras, arquitetura e estado.

PRIORIDADE REAL

Mao é o bloqueador imediato. Enquanto sua lista existir apenas como variável local, não há lugar persistente para receber uma carta retirada do baralho.

Grafo de objetos atual



a lista criada no construtor não é guardada

O desenho representa referências conservadas, não apenas objetos construídos.

Figura 1. Referências relevantes conservadas depois dos construtores.

O desenho diferencia construir um objeto de conservar uma referência. Dealer possui um campo mao, mas a lista interna de Mao ainda é local. Jogador cria uma Mao dentro do construtor e perde a referência ao terminar. Blackjack, por sua vez, cria objetos extras nos inicializadores de campo e depois os substitui pelos argumentos do construtor.

Débito de inicialização em Blackjack

ESTADO ATUAL RESUMIDO

```
String nome; // começa como null
Baralho baralho = new Baralho(); // cria 52 cartas extras
Jogador jogador = new Jogador(nome); // jogador temporário
Dealer dealer = new Dealer(); // dealer temporário

// Depois o construtor substitui todos eles pelos argumentos.
```

CONSEQUÊNCIA CONCRETA

Ao iniciar uma partida, 104 cartas chegam a ser construídas: 52 no Baralho criado por App e 52 no Baralho temporário de Blackjack. Os objetos temporários ficam elegíveis para coleta de lixo depois de serem substituídos, mas a criação é desnecessária.

CAPÍTULO 02

02

Escopo do Blackjack mínimo

Antes dos métodos, congele as regras. Sem esse acordo, cada implementação resolve um jogo diferente.

MVP recomendado

- ☐ Um jogador humano contra uma banca.
- ☐ Um baralho padrão com 52 cartas e sem curingas.
- ☐ Distribuição inicial de duas cartas para cada participante.
- ☐ Jogador pode pedir carta ou parar.
- ☐ Banca compra abaixo de 17 e para em 17 ou mais.
- ☐ Para esta primeira versão, a banca para também em soft 17.
- ☐ Pontuação trata figuras e múltiplos ases.
- ☐ Resultado distingue vitória, derrota, empate, blackjack natural e estouro.
- ☐ Ao final, o usuário pode iniciar outra rodada ou sair.

Explicitamente fora do primeiro ciclo

- Apostas e fichas.
- Split, double down, seguro e surrender.
- Múltiplos jogadores.
- Vários baralhos e carta de corte.
- Persistência de saldo ou histórico.
- Interface gráfica.

POR QUE LIMITAR?

Cada regra avançada amplia o número de estados e decisões. Um núcleo pequeno permite aprender responsabilidades, referências, coleções e testes antes de multiplicar exceções.

Vocabulário de regras

Termo	Definição adotada	Consequência no código
Blackjack natural	21 com exatamente duas cartas: ás e carta de valor-base 10.	Não confundir com 21 formado por três ou mais cartas.
Estouro	Pontuação total maior que 21.	Encerra o turno daquele participante.
Mão macia	Ao menos um ás ainda está sendo contado como 11.	Importa para a política da banca.
Soft 17	Total 17 com um ás ainda valendo 11.	Neste MVP, a banca para.
Empate	Totais iguais sem regra especial que determine vencedor.	Resultado próprio, não vitória silenciosa da banca.
Topo do baralho	Uma extremidade escolhida da lista.	Defina uma vez; retirar do fim de ArrayList é simples.

Tabela de decisões do projeto

- ☐ Registrar no README: banca para em soft 17.
- ☐ Registrar no README: não há apostas nesta versão.
- ☐ Definir o comportamento de comprar quando o baralho está vazio.
- ☐ Definir se uma nova rodada cria um novo baralho ou reinicia o existente.
- ☐ Definir quando a segunda carta da banca fica visível.
- ☐ Definir o nome do enum de resultado da rodada.

PARE E PENSE

Uma regra não documentada vira uma decisão escondida dentro de um `if`. Que pessoa conseguiria descobrir a variante escolhida lendo apenas o README e os testes?

CAPÍTULO 03

03

Mapa de dependências

A sequência correta reduz retrabalho: primeiro o estado existe, depois as cartas se movem, então as regras coordenam o movimento.

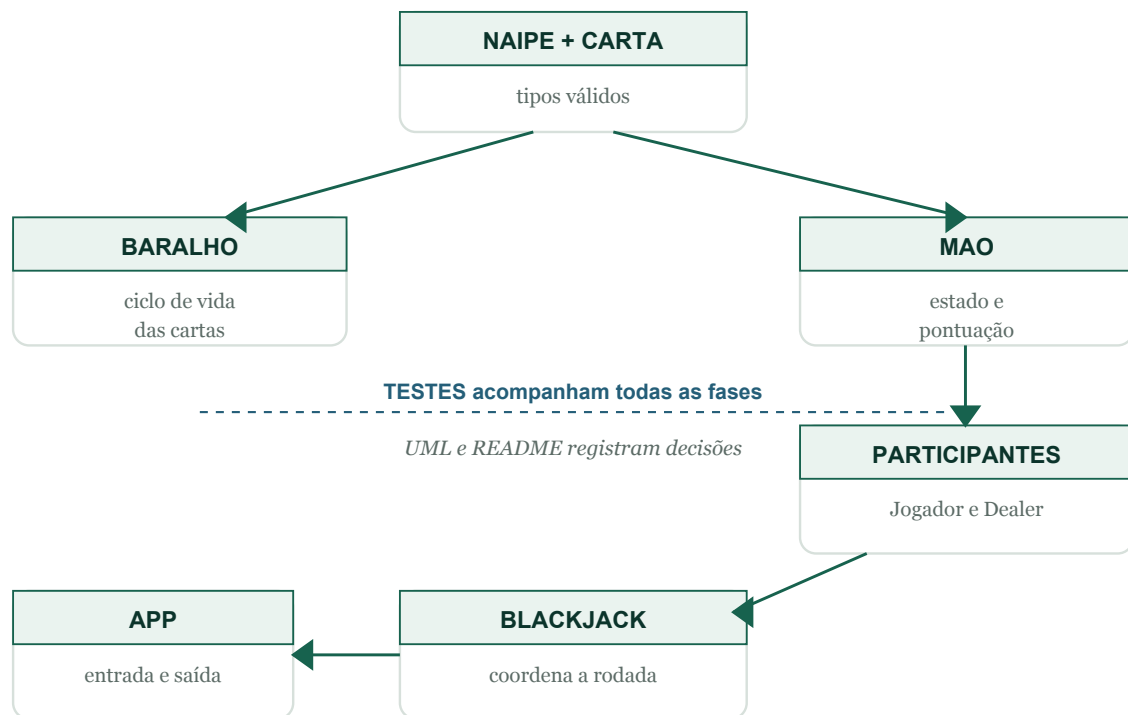


Figura 2. Dependências de implementação e trilhas transversais.

Ordem recomendada

1. Preservar estado em Mao, Jogador e Dealer.
2. Fechar o conjunto de valores possíveis de Carta.
3. Dar a Baralho operações para embaralhar e entregar cartas.
4. Limpar a inicialização de Blackjack e distribuir as quatro cartas iniciais.
5. Calcular pontuação e situações especiais em Mao.
6. Modelar turnos e resultado em Blackjack.
7. Conectar o domínio à interface de App.
8. Manter testes, PUM e README sincronizados em cada marco.

Por que não começar pelo menu?

O menu consegue perguntar 'pedir ou parar', mas ainda não existe uma mão que guarde a carta, um baralho que a retire, nem um jogo que coordene a transferência. Começar pela interface obrigaria App a assumir regras que depois precisariam ser movidas.

REGRA DE DEPENDÊNCIA

Uma camada deve conversar com comportamentos estáveis da camada anterior. App depende de Blackjack; Blackjack depende de Baralho, participantes e mãos; mãos e baralho dependem de Carta.

CAPÍTULO F0

F0

Ler o sistema antes de alterá-lo

Depois que um construtor termina, quem ainda aponta para cada objeto criado?

OBJETIVO DA FASE

Construir um modelo mental preciso de classe, objeto, referência, campo e variável local antes de adicionar regras.

O que o código atual revela

- Mao cria `ArrayList<Carta> mao` dentro do construtor; essa variável não é um campo.
- Jogador cria `Mao mao` dentro do construtor e não conserva a referência.
- Dealer declara `Mao mao` diretamente no corpo da classe; portanto conserva a referência.
- Blackjack mistura dependências recebidas com objetos construídos antecipadamente.

Conceitos que você está praticando

- Classe versus instância.
- Variável de referência versus objeto referenciado.
- Escopo léxico e alcançabilidade.
- Campos como estado persistente.
- Construtor como estabelecimento de invariantes.
- Coleta de lixo sem a simplificação 'apagado imediatamente'.

PARE E PENSE

Depois que um construtor termina, quem ainda aponta para cada objeto criado?

Decisões antes do código

1. Marque cada declaração atual como campo, parâmetro ou variável local.
2. Desenhe o grafo de referências após `new Blackjack(...)`.
3. Liste quais objetos são obrigatórios para um jogo válido.
4. Decida quais construtores criam dependências e quais as recebem.

Missão prática

- ☐ Anotar, no próprio caderno, o escopo de cada variável relevante.
- ☐ Prever a saída e o grafo antes de executar o programa.
- ☐ Identificar todos os campos `package-private` que deveriam proteger estado.
- ☐ Explicar por que `final List<Carta>` não torna a lista imutável.
- ☐ Registrar a baseline: 3 testes passando.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Construtor	Deixar o objeto válido ao terminar.	Executar a partida inteira ou esconder efeitos inesperados.
Campo	Conservar estado necessário depois do método.	Virar público apenas para facilitar um teste.
Variável local	Apoiar um cálculo temporário.	Representar estado que o objeto precisa lembrar.

Testes-alvo

Cenário	Evidência observável
Mao recém-criada	A API pública deve revelar quantidade zero quando for implementada.
Duas referências para a mesma Mao	Alterações pelo primeiro nome são observadas pelo segundo.
Duas instâncias de Mao	Alterar uma não altera a outra.

Armadilhas previsíveis

- Dizer que a variável contém o objeto; ela contém uma referência que permite alcançá-lo.
- Dizer que sair do escopo destrói o objeto imediatamente.
- Transformar campos em `static` para 'não perder' o valor.
- Criar getters públicos para toda coleção apenas para inspecioná-la.

Pronto quando

- ☐ Você consegue classificar campos, parâmetros e locais sem executar o código.
- ☐ Você explica por que a lista de Mao se torna inalcançável.
- ☐ Você identifica os objetos redundantes de Blackjack.
- ☐ Nenhuma regra do jogo foi adicionada antes de entender o estado.

Explique em voz alta

- Qual é a diferença entre construir e armazenar?
- Por que Dealer conserva Mao e Jogador não?
- O que `this` seleciona?
- Por que Java passa uma cópia do valor da referência?

CAPÍTULO F1

F1

Fazer a mão e os participantes conservarem estado

Onde a coleção deve viver para continuar existindo depois do construtor?

OBJETIVO DA FASE

Criar o primeiro grafo de objetos estável: cada participante possui uma mão própria, e cada mão conserva suas cartas.

O que o código atual revela

- Mao termina o construtor sem qualquer campo.
- Jogador conserva o nome, mas a Mao criada é apenas local.
- Dealer conserva uma Mao, porém o campo não é privado nem final.
- Os imports de ArrayList em Jogador e Dealer não são usados.

Conceitos que você está praticando

- Composição: participante tem uma mão.
- Estado por instância; perigo de compartilhar com `static`.
- Programar para a interface `List<Carta>`.
- Encapsulamento por comportamento, não por getters/setters automáticos.
- Aliasing e cópia defensiva de coleções.

PARE E PENSE

Onde a coleção deve viver para continuar existindo depois do construtor?

Decisões antes do código

1. Escolha o nome do campo da coleção em Mao: evite chamar a coleção e a classe pelo mesmo conceito sem clareza.
2. Decida quais operações mínimas provam que a mão conserva estado.
3. Decida se Jogador e Dealer criam a própria Mao ou a recebem no construtor.
4. Defina a invariante do nome do jogador.
5. Escolha como inspecionar cartas sem devolver a lista mutável original.

Missão prática

- ☐ Criar em Mao um campo privado e final do tipo `List<Carta>`.
- ☐ Inicializar a coleção no construtor.
- ☐ Criar uma operação para receber uma carta não nula.
- ☐ Criar uma consulta de quantidade.
- ☐ Fazer Jogador conservar sua Mao em campo.
- ☐ Encapsular e validar o nome.
- ☐ Tornar a Mao de Dealer privada e final.
- ☐ Remover imports e construtores vazios que não agregam comportamento.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Mao.adicionar(Carta)	Conservar uma carta válida na coleção interna.	Buscar a carta no baralho ou imprimir mensagens.
Mao.getQuantidadeCartas()	Informar o tamanho atual.	Expor a implementação concreta da coleção.
Jogador.getMao()	Permitir colaboração controlada com a mão.	Trocar a mão por meio de setter.
Jogador.getNome()	Informar o nome validado.	Alterar o nome como efeito colateral.
Dealer.getMao()	Dar acesso ao comportamento da própria mão.	Compartilhar uma mão global.

Testes-alvo

Cenário	Evidência observável
Nova Mao	Quantidade igual a zero.
Adicionar uma carta	Quantidade aumenta para um e a referência recebida é conservada.
Adicionar null	A mão rejeita segundo o contrato escolhido.
Duas mãos	Adicionar em uma não altera a outra.
Jogador e Dealer	Cada um possui uma instância diferente de Mao.
Nome vazio	Jogador inválido não nasce ou a política escolhida é explícita.

Armadilhas previsíveis

- Declarar a lista dentro do construtor novamente.
- Usar `static List<Carta>` e fazer todas as mãos compartilharem cartas.
- Devolver a lista interna mutável para App chamar `add` diretamente.
- Guardar quantidade e lista separadamente, permitindo divergência.
- Criar herança entre Jogador e Dealer antes de existir comportamento comum significativo.

Pronto quando

- ☐ Uma carta adicionada ainda pode ser consultada depois que o método termina.
- ☐ Cada participante conserva exatamente sua própria mão.
- ☐ Campos de estado estão privados e referências obrigatórias estão finais.
- ☐ Os testes usam API pública e não reflexão para alcançar campos.

Explique em voz alta

- Por que a lista precisa ser campo, mas a variável temporária de um cálculo não?
- O que `final` impede em uma lista e o que não impede?
- Por que receber carta pertence à Mao?
- Que problema surgiria se duas mãos compartilhassem a mesma lista?

CAPÍTULO F2

F2

Fechar o domínio de Carta

Se naipe é um conjunto fechado, por que valor nominal ainda aceita qualquer texto?

OBJETIVO DA FASE

Impedir que cartas impossíveis nasçam e separar rank, valor-base e pontuação contextual.

O que o código atual revela

- Carta rejeita null e texto em branco, mas aceita XYZ, 14 ou -8.
- Naipe já demonstra como um enum substitui Strings livres.
- Baralho conhece os 13 ranks por meio de um array local.
- A identidade Q não deve ser substituída pela String 10 apenas porque sua pontuação-base é dez.

Conceitos que você está praticando

- Enum como tipo de domínio.
- Objeto de valor imutável.
- Identidade lógica versus identidade de referência.
- Valor-base versus valor derivado pelo contexto.
- Contrato de equals e hashCode.

PARE E PENSE

Se naipe é um conjunto fechado, por que valor nominal ainda aceita qualquer texto?

Decisões antes do código

1. Escolha um nome: `ValorCarta` ou `Rank`.
2. Defina as 13 constantes sem usar `ordinal()` como pontuação.
3. Decida se o enum conhece apenas o valor-base ou também perguntas como `ehAs()`.
4. Decida se duas cartas de mesmo rank e naipe são logicamente iguais neste modelo de um baralho.
5. Defina uma representação textual legível sem depender de símbolos incertos.

Missão prática

- ☐ Criar o enum dos 13 valores.
- ☐ Alterar `Carta` para receber o novo tipo em vez de `String`.
- ☐ Atualizar `Baralho` para percorrer os valores válidos.
- ☐ Manter `Carta` sem setters e com campos finais.
- ☐ Adicionar representação textual útil para terminal e testes.
- ☐ Se a decisão for objeto de valor, implementar igualdade e hash code coerentes.
- ☐ Atualizar testes antes de remover todas as `Strings`.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
<code>ValorCarta.getValorBase()</code>	Informar 2-10 e valor-base de figuras/ás.	Decidir sozinho se o ás vale 1 ou 11 na mão.
<code>Carta.getValorNominal()</code>	Informar o rank tipado da carta.	Converter Q em 10 e perder a identidade.
<code>Carta.toString()</code>	Produzir texto legível para interface e diagnóstico.	Imprimir diretamente no console como efeito colateral.
<code>Carta.equals(Object)</code>	Aplicar a igualdade lógica escolhida.	Usar campos diferentes dos usados por <code>hashCode</code> .

Testes-alvo

Cenário	Evidência observável
Todos os 13 ranks	Podem formar Carta com cada Naipes.
Rank inexistente	Não pode ser representado pelo tipo.
Figura	Continua sendo J, Q ou K e possui valor-base 10.
Ás isolado	Mantém identidade A; a mão decide sua interpretação.
Duas cartas equivalentes	equals e hashCode obedecem à política documentada.
Texto da carta	É legível e inclui rank e naipe.

Armadilhas previsíveis

- Usar `ordinal()` como valor; reordenar o enum mudaria a regra.
- Manter simultaneamente `String` e `enum` como duas fontes de verdade.
- Colocar a lógica completa de múltiplos ases dentro de `Carta`.
- Implementar `equals` sem `hashCode`.
- Dar setters a `Carta` para reaproveitar o mesmo objeto durante a montagem do baralho.

Pronto quando

- ☐ Nenhuma carta inválida pode nascer.
- ☐ Carta conserva rank e naipe durante toda a vida.
- ☐ Figuras não perdem sua identidade.
- ☐ Baralho não mantém uma lista paralela de `Strings` válidas.
- ☐ Os testes descrevem a política de igualdade.

Explique em voz alta

- Por que `Naipes.COPAS` é diferente da `String 'COPAS'`?
- Por que o ás exige contexto de `Mao`?
- O que significa duas cartas serem iguais neste projeto?
- Qual mudança futura quebraria um uso de `ordinal`?

CAPÍTULO F3

F3

Completar o ciclo de vida do Baralho

Que operação garante que comprar uma carta diminui exatamente uma unidade?

OBJETIVO DA FASE

Fazer o Baralho proteger suas cartas restantes, embaralhar a própria coleção e transferir uma carta por vez.

O que o código atual revela

- Baralho cria 52 cartas corretas e informa a quantidade.
- Ainda não existe operação para retirar uma carta.
- Blackjack possui um método estático e vazio chamado embaralhar, embora as cartas pertençam ao Baralho.
- O conteúdo interno não é exposto, o que preserva encapsulamento, mas exige APIs de domínio para testes e colaboração.

Conceitos que você está praticando

- Método de instância versus método estático.
- Mutação interna controlada.
- Remoção e transferência de referência.
- Política de coleção vazia.
- Aleatoriedade determinística em testes.

PARE E PENSE

Que operação garante que comprar uma carta diminui exatamente uma unidade?

Decisões antes do código

1. Escolha qual extremidade representa o topo.
2. Escolha entre lançar exceção clara ou devolver Optional quando vazio.
3. Decida se embaralhar recebe um Random para testes ou usa uma versão simples inicialmente.
4. Defina uma forma segura de verificar composição sem entregar a lista mutável.
5. Defina se um Baralho pode ser reiniciado ou se uma nova instância representa uma nova rodada.

Missão prática

- ☐ Mover a responsabilidade de embaralhar para Baralho.
- ☐ Criar uma operação que remova e devolva uma carta.
- ☐ Criar consulta para baralho vazio.
- ☐ Preservar a lista como detalhe privado.
- ☐ Testar as 52 combinações, não somente o tamanho.
- ☐ Testar esgotamento até a última carta.
- ☐ Documentar o comportamento da 53a compra.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Baralho.embaralhar()	Reordenar as cartas restantes da própria instância.	Criar outro baralho ou alterar cartas já entregues.
Baralho.comprar()	Remover e devolver uma única carta.	Devolver sem remover ou retornar null silenciosamente.
Baralho.estaVazio()	Responder a partir do tamanho atual.	Manter um boolean separado que possa ficar desatualizado.
Baralho.getQuantidadeCartas()	Informar quantas ainda estão disponíveis.	Expor a lista interna.

Testes-alvo

Cenário	Evidência observável
Baralho novo	52 cartas e 52 combinações únicas.
Distribuição por naipe	13 cartas de cada Naipes.
Distribuição por rank	Quatro cartas de cada ValorCarta.
Uma compra	Quantidade passa de 52 para 51.
Esgotamento	52 compras não repetem uma mesma carta.
Compra vazio	O contrato escolhido é obedecido.
Embaralhamento	Quantidade e conjunto são preservados.

Armadilhas previsíveis

- Testar que embaralhar sempre muda a ordem; a mesma permutação é possível.
- Usar `remove(0)` sem perceber o deslocamento de `ArrayList`; para 52 cartas não é grave, mas a escolha deve ser consciente.
- Devolver a lista para Blackjack remover diretamente.
- Clonar uma Carta na compra e deixar a original no baralho.
- Retornar null e espalhar verificações tardias.

Pronto quando

- ☐ Comprar remove exatamente uma referência.
- ☐ A lista interna continua inacessível para mutação externa.
- ☐ O comportamento de vazio é explícito e testado.
- ☐ Embaralhar pertence ao Baralho.
- ☐ Os testes detectariam um baralho com 52 ases iguais.

Explique em voz alta

- Por que embaralhar não faz sentido sem escolher uma instância de Baralho?
- Qual objeto conhece as cartas restantes?
- Como provar que nenhuma carta foi duplicada?
- O que deveria acontecer na compra 53?

MODELO VISUAL

A transferência de uma carta



comprar() remove e devolve; adicionar() conserva a referência

A carta não deve continuar simultaneamente nas duas coleções.

Figura 3. Uma referência sai do baralho e passa a ser conservada pela mão.

A colaboração recomendada é uma sequência de mensagens: Blackjack pede uma carta ao Baralho; Baralho remove e devolve; Blackjack entrega à Mão; Mão conserva. A mão não conhece o baralho e o baralho não conhece participantes.

Invariante de conservação

CONTA DAS CARTAS

Durante uma rodada sem descarte, a soma **cartas restantes no baralho + cartas nas mãos** deve permanecer 52. Depois da distribuição inicial para um jogador e uma banca: $48 + 2 + 2 = 52$.

Experimento de referência

1. Guarde a carta retornada por comprar em uma variável local.
2. Adicione exatamente essa referência a uma Mão.
3. Verifique que o tamanho do baralho diminuiu e o da mão aumentou.
4. Se existir uma política de igualdade, diferencie `assertSame` de `assertEquals`.
5. Explique por que `addAll` ou `add` copia referências, não clona objetos.

CAPÍTULO F4

F4

Transformar Blackjack em coordenador

O construtor deve criar a partida ou apenas receber e validar seus participantes?

OBJETIVO DA FASE

Eliminar inicializações redundantes e fazer Blackjack coordenar objetos por comportamentos públicos.

O que o código atual revela

- Blackjack recebe Baralho, Jogador e Dealer, mas cria versões temporárias antes do corpo do construtor.
- O campo nome permanece null e duplica uma informação que pertence ao Jogador.
- Os campos têm visibilidade de pacote e não são finais.
- O método embaralhar está vazio, estático e na classe errada.
- App cria a variável jogo, mas não chama nenhuma operação.

Conceitos que você está praticando

- Injeção de dependência manual.
- Construtor e invariantes.
- Orquestração versus posse de dados.
- Baixo acoplamento por mensagens.
- Efeitos explícitos.

PARE E PENSE

O construtor deve criar a partida ou apenas receber e validar seus participantes?

Decisões antes do código

1. Escolha definitivamente: Blackjack recebe suas dependências ou as cria; a recomendação é recebê-las.
2. Defina quais dependências jamais podem ser null.
3. Decida qual método inicia uma rodada.
4. Defina se a distribuição inicial alterna jogador e banca.
5. Planeje como testar com uma ordem de cartas previsível.

Missão prática

- ☐ Remover o campo nome.
- ☐ Remover inicializadores que constroem Baralho, Jogador e Dealer.
- ☐ Tornar as três dependências privadas e finais.
- ☐ Validar dependências no construtor.
- ☐ Criar uma operação explícita de distribuição inicial.
- ☐ Pedir cartas por meio de Baralho.comprar().
- ☐ Entregar cartas por meio do comportamento de Mao.
- ☐ Remover o método embaralhar de Blackjack.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Blackjack(...)	Conservar dependências obrigatórias válidas.	Criar substitutos temporários ou começar a rodada silenciosamente.
distribuirCartasIniciais()	Transferir duas cartas para cada participante.	Manipular as listas internas diretamente.
iniciarRodada()	Coordenar preparação segundo a sequência definida.	Ler Scanner ou imprimir interface dentro das entidades.

Testes-alvo

Cenário	Evidência observável
Construção	Não cria um segundo baralho nem participantes secretos.
Dependência null	Construção falha perto da causa.
Distribuição inicial	Jogador recebe duas e Dealer recebe duas.
Conservação	Baralho termina com 48 cartas.
Ordem	Com baralho preparado, a alternância é previsível.
Identidade	As cartas entregues são as mesmas retiradas.

Armadilhas previsíveis

- Fazer o construtor executar toda a partida.
- Criar novos participantes dentro de distribuir e ignorar os campos.
- Pedir a lista interna do Baralho para removê-la externamente.
- Misturar Scanner e System.out com regras de domínio.
- Testar distribuição com embaralhamento real e aceitar testes intermitentes.

Pronto quando

- ☐ Construir Blackjack não aloca outro baralho.
- ☐ As dependências recebidas permanecem as utilizadas.
- ☐ A distribuição inicial é explícita e determinística em teste.
- ☐ Depois da distribuição, $48 + 2 + 2 = 52$.
- ☐ Blackjack coordena; Baralho e Mao protegem seus próprios estados.

Explique em voz alta

- Por que receber dependências facilita testes?
- Qual é a diferença entre coordenar uma transferência e manipular duas listas?
- Por que nome pertence ao Jogador?
- Que efeitos devem acontecer apenas quando iniciarRodada for chamado?

CAPÍTULO F5

F5

Calcular pontuação no lugar correto

Por que uma Carta isolada não consegue decidir se o ás vale 1 ou 11?

OBJETIVO DA FASE

Fazer Mao interpretar o conjunto de cartas sem alterar a identidade de nenhuma Carta.

O que o código atual revela

- Carta conserva identidade e naipe, mas ainda não oferece valor-base tipado.
- Mao ainda não calcula total, estouro ou blackjack.
- O ás depende das outras cartas; sua interpretação é contextual.
- Armazenar um campo total duplicaria informação derivável da coleção.

Conceitos que você está praticando

- Valor derivado.
- Responsabilidade baseada em contexto.
- Algoritmo para múltiplos ases.
- Mão macia.
- Blackjack natural versus total 21.

PARE E PENSE

Por que uma Carta isolada não consegue decidir se o ás vale 1 ou 11?

Decisões antes do código

1. Decida se ValorCarta informa o valor-base do ás como 11.
2. Defina a regra: somar ases como 11 e ajustar enquanto necessário.
3. Defina a política de blackjack natural.
4. Decida se o total será sempre recalculado ou terá cache cuidadosamente invalidado; recomendo recalculá-lo agora.
5. Defina nomes claros para estourou, temVinteEUm, temBlackjackNatural e ehMacia.

Missão prática

- ☐ Somar cartas numéricas normalmente.
- ☐ Somar J, Q e K como 10.
- ☐ Contar inicialmente cada ás como 11.
- ☐ Enquanto o total exceder 21 e houver ás flexível, subtrair 10.
- ☐ Criar consultas derivadas sem campos booleanos duplicados.
- ☐ Não alterar Carta durante o cálculo.
- ☐ Cobrir vários ases em testes separados.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Mao.calcularPontuacao()	Derivar o melhor total possível das cartas atuais.	Alterar o rank ou guardar total desatualizado.
Mao.estourou()	Responder se o total é maior que 21.	Manter um campo independente do cálculo.
Mao.temBlackjackNatural()	Verificar total 21 com exatamente duas cartas.	Chamar todo total 21 de blackjack.
Mao.ehMacia()	Indicar se algum ás ainda conta como 11.	Decidir a política da banca.

Testes-alvo

Cenário	Evidência observável
10 + Q	Total 20.
A + 9	Total 20 e mão macia.
A + K	Total 21 e blackjack natural.
A + A	Total 12.
A + A + 9	Total 21.
A + 6 + 10	Total 17 após ajustar o ás.
A + A + A + 8	Total 21.
7 + 7 + 7	Total 21, mas não blackjack natural.
K + Q + 2	Total 22 e estouro.

Armadilhas previsíveis

- Transformar o campo do ás de 11 para 1 durante o cálculo.
- Ajustar apenas um ás e falhar com três ases.
- Guardar o total e esquecer de atualizá-lo ao receber carta.
- Colocar a lógica completa em Carta.
- Tratar 21 com três cartas como blackjack natural.

Pronto quando

- ☐ Todos os cenários de ás passam.
- ☐ O cálculo não modifica as cartas.
- ☐ O total sempre reflete a coleção atual.
- ☐ Blackjack natural exige exatamente duas cartas.
- ☐ Mão macia é distinguível para a política da banca.

Explique em voz alta

- Qual objeto vê todas as cartas necessárias para ajustar o ás?
- Por que pontuação é valor derivado?
- Em A + A + 9, quantos ases ainda podem valer 11?
- Por que 7 + 7 + 7 não é blackjack natural?

LABORATÓRIO DE REGRAS

Teste manual de múltiplos ases

Mão	Total	Macia?	Raciocínio
A + 6	17	Sim	Um ás permanece 11.
A + 9	20	Sim	Um ás permanece 11.
A + K	21	Sim	Natural com duas cartas.
A + A	12	Sim	11 + 1.
A + A + 9	21	Sim	11 + 1 + 9.
A + 6 + 10	17	Não	O ás precisa cair de 11 para 1.
A + A + 10 + K	22	Não	Mesmo com ambos valendo 1, estoura.
A + A + A + 8	21	Sim	11 + 1 + 1 + 8.

Ficha de previsão

- ☐ Antes de executar, escrevi o total esperado.
- ☐ contei quantos ases foram inicialmente somados como 11.
- ☐ Ajustei um ás por vez somente enquanto o total excedia 21.
- ☐ Verifiquei separadamente total, estouro, mão macia e blackjack natural.
- ☐ Se o teste falhou, descrevi qual regra estava ausente.

INVARIANTE ÚTIL

A pontuação escolhida deve ser a maior soma válida que não ultrapassa 21; se todas ultrapassarem, deve ser a menor soma possível após ajustar todos os ases necessários.

CAPÍTULO F6

F6

Modelar turnos e resultado da rodada

Que ações são válidas em cada momento e quais transições precisam ser impossíveis?

OBJETIVO DA FASE

Evitar um método monolítico modelando a rodada como uma sequência explícita de estados e decisões.

O que o código atual revela

- Não existe ainda turno do jogador, turno da banca ou cálculo de resultado.
- O jogo retorna ao menu imediatamente depois da construção.
- As regras de soft 17, empate e natural ainda não estão documentadas.
- App não deveria decidir vencedor com condicionais de interface.

Conceitos que você está praticando

- Máquina de estados.
- Comando versus consulta.
- Enum de estado e enum de resultado.
- Pré-condições de operações.
- Política da banca separada da interface.

PARE E PENSE

Que ações são válidas em cada momento e quais transições precisam ser impossíveis?

Decisões antes do código

1. Escolha os estados mínimos da rodada.
2. Defina eventos que avançam cada estado.
3. Defina a ordem de términos antecipados.
4. Confirme a política: banca para em soft 17 neste MVP.
5. Defina como empate e blackjack natural são comparados.
6. Decida se pedir após parada lança erro ou é simplesmente impossível pela API.

Missão prática

- ☐ Representar estados e resultados com nomes explícitos.
- ☐ Criar operação para o jogador pedir uma carta.
- ☐ Criar operação para registrar que o jogador parou.
- ☐ Executar o turno automático da banca.
- ☐ Interromper imediatamente quando o jogador estoura.
- ☐ Determinar resultado sem imprimir.
- ☐ Impedir ações depois de FINALIZADA.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
pedirCartaJogador()	Transferir uma carta quando o estado permite.	Ler uma opção do Scanner.
pararJogador()	Encerrar decisões do jogador e avançar o estado.	Executar toda a banca dentro de um setter.
executarTurnoDealer()	Comprar segundo a política documentada.	Consultar entrada humana.
determinarResultado()	Comparar mãos segundo precedência definida.	Imprimir texto ou alterar cartas.
getEstado()	Informar a fase atual.	Permitir alteração externa do estado.

Testes-alvo

Cenário	Evidência observável
Jogador pede	Baralho diminui e mão aumenta quando estado permite.
Jogador estoura	Rodada finaliza sem compras desnecessárias da banca.
Jogador para	Estado avança para turno da banca.
Banca abaixo de 17	Compra outra carta.
Banca em soft 17	Para, conforme regra adotada.
Banca estoura	Jogador vence se não estiver estourado.
Totais iguais	Resultado EMPATE.
A + K contra 7 + 7 + 7	Blackjack natural recebe a precedência definida.
Rodada finalizada	Nova compra é rejeitada ou inacessível.

Armadilhas previsíveis

- Criar um único método de centenas de linhas que lê entrada, compra e imprime.
- Usar Strings livres como 'jogando', 'fim', 'parou'.
- Permitir que App altere o estado por setter.
- Executar o turno da banca mesmo após estouro do jogador.
- Esconder a política de soft 17 em uma condição sem teste.

Pronto quando

- ☐ Cada operação declara em qual estado é válida.
- ☐ Todas as transições importantes têm teste.
- ☐ O resultado é um valor de domínio, não texto impresso.
- ☐ A banca segue a política documentada.
- ☐ Nenhuma compra ocorre depois de finalizar.

Explique em voz alta

- Qual estado permite ao jogador pedir?
- Qual evento encerra o turno humano?
- Em que ordem verificar natural, estouro e comparação?
- Por que App não deve decidir o vencedor?

MODELO VISUAL

Máquina de estados mínima

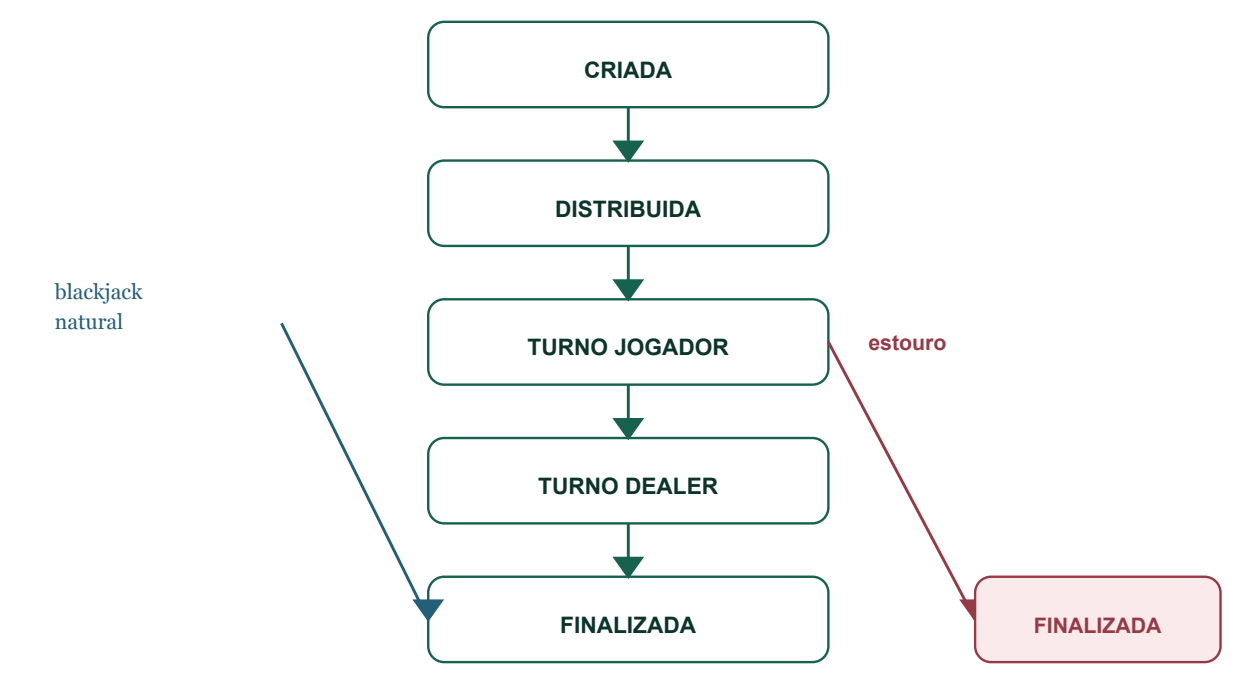


Figura 4. Fluxo principal e dois términos antecipados.

Tabela de transições para completar

Estado atual	Evento	Condição	Próximo estado
CRIADA	iniciar	Dependências válidas.	CARTAS_DISTRIBUIDAS
CARTAS_DISTRIBUIDAS	avaliar naturais	Sem término antecipado.	TURNO_DO_JOGADOR
TURNO_DO_JOGADOR	pedir	Não estourou e não parou.	Permanece ou FINALIZADA
TURNO_DO_JOGADOR	parar	Ação válida.	TURNO_DA_BANCA
TURNO_DA_BANCA	comprar/parar	Política de 17.	Permanece ou FINALIZADA
FINALIZADA	consultar resultado	Resultado calculado.	FINALIZADA

EXERCÍCIO

Acrescente à tabela os términos por blackjack natural e por estouro. Depois escreva um teste para cada transição antes de implementar seu corpo.

CAPÍTULO F7

F7

Conectar o domínio à interface

Como App pode conduzir a rodada sem receber listas internas nem recalculas regras?

OBJETIVO DA FASE

Fazer a interface cuidar apenas de entrada, saída e tradução das escolhas do usuário para mensagens ao jogo.

O que o código atual revela

- O menu não mostra o, embora o laço encerre com o.
- Opções 2, 3 e 4 aparecem, mas são ignoradas.
- `next ( )` lê apenas um token; nomes com espaços deixam conteúdo que quebra a próxima leitura numérica.
- Entrada não numérica provoca exceção.
- O objeto jogo é criado e imediatamente abandonado sem receber uma operação.
- O nome `wel lcomeBlackjack` está grafado incorretamente e mistura idiomas.

Conceitos que você está praticando

- Separação entre interface e domínio.
- Parsing e validação de entrada.
- Loop de interação.
- Switch sem fall-through.
- Testabilidade sem Scanner.

PARE E PENSE

Como App pode conduzir a rodada sem receber listas internas nem recalculas regras?

Decisões antes do código

1. Decida se o menu mostrará apenas Blackjack e sair durante o MVP.
2. Escolha uma estratégia: ler linhas completas e converter a opção.
3. Defina mensagens para opção inválida e jogo ainda não implementado.
4. Defina quais dados de estado Blackjack expõe para apresentação.
5. Planeje o ciclo: iniciar, mostrar mãos, pedir/parar, resultado, repetir/sair.

Missão prática

- ☐ Mostrar explicitamente 0) Sair.
- ☐ Tratar texto não numérico sem encerrar o programa.
- ☐ Aceitar nome completo e rejeitar nome em branco.
- ☐ Adicionar comportamento para opções indisponíveis ou removê-las do menu.
- ☐ Chamar a operação que inicia a rodada.
- ☐ Traduzir pedir/parar em mensagens ao objeto Blackjack.
- ☐ Exibir resultado retornado pelo domínio.
- ☐ Manter cálculo de pontos e vencedor fora de App.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
lerOpcao()	Converter uma linha válida em opção.	Executar regras de Blackjack.
playBlackjack(nome)	Montar dependências e conduzir a interface.	Calcular pontuação diretamente.
mostrarEstado(jogo)	Apresentar dados públicos do domínio.	Receber a lista mutável de cartas.
main	Controlar o ciclo da aplicação.	Virar um objeto global compartilhado por static.

Testes-alvo

Cenário	Evidência observável
Entrada o	Programa encerra de forma clara.
Entrada não numérica	Mensagem útil e nova tentativa.
Nome com espaços	Nome completo é conservado.
Opção indisponível	Usuário recebe informação, sem silêncio.
Rodada completa	É possível pedir/parar e ver resultado.
Nova rodada	Estado anterior não vaza para a próxima.

Armadilhas previsíveis

- Misturar `nextInt()`, `next()` e `nextLine()` sem entender o buffer.
- Duplicar o cálculo de pontos em App para imprimir.
- Deixar opções inexistentes no menu sem resposta.
- Fechar `System.in` dentro de um método auxiliar.
- Usar variáveis `static` para compartilhar jogo entre métodos.

Pronto quando

- ☐ Toda saída real tem uma opção visível.
- ☐ Entrada inválida não encerra o processo.
- ☐ Nome completo é aceito conforme a decisão.
- ☐ App chama comportamentos; não manipula coleções.
- ☐ Uma rodada manual termina e pode ser repetida.

Explique em voz alta

- Que parte do programa conhece Scanner?
- Que parte sabe quem venceu?
- Como evitar que um nome com espaço quebre a próxima opção?
- Quais informações a interface precisa consultar sem quebrar encapsulamento?

CAPÍTULO F8

F8

Usar testes, UML e README como especificação

Se a implementação mudar amanhã, quais artefatos revelarão imediatamente uma regressão ou divergência?

OBJETIVO DA FASE

Transformar verificação e documentação em trilhas permanentes, não em tarefas deixadas para o final.

O que o código atual revela

- Os três testes atuais cobrem quantidade inicial, conservação/normalização da Carta e naipe nulo.
- Todos os testes ainda estão em AppTest.
- O PUML omite delimitadores e descreve métodos removidos ou em classes erradas.
- README documenta comandos Maven, mas não o domínio ou o estado de implementação.

Conceitos que você está praticando

- Arrange, Act, Assert.
- Teste de comportamento público.
- Determinismo e aleatoriedade controlada.
- UML descritivo versus modelo-alvo.
- Documentação de decisões.

PARE E PENSE

Se a implementação mudar amanhã, quais artefatos revelarão imediatamente uma regressão ou divergência?

Decisões antes do código

1. Defina uma classe de teste por unidade principal.
2. Escreva cenários com nome que descreve causa e resultado.
3. Não teste métodos privados diretamente; prove seus efeitos pela API.
4. Mantenha dois diagramas enquanto necessário: estado atual e alvo.
5. Atualize README ao decidir uma regra, não meses depois.

Missão prática

- ☐ Criar CartaTest, BaralhoTest, MaoTest, JogadorTest e BlackjackTest conforme surgirem comportamentos.
- ☐ Escrever teste antes ou junto da menor implementação.
- ☐ Usar Dado/Quando/Então para cenários de domínio.
- ☐ Corrigir @startuml e @enduml.
- ☐ Remover do PUMML métodos que não existem.
- ☐ Adicionar Naipes e o futuro ValorCarta.
- ☐ Documentar escopo e soft 17 no README.
- ☐ Executar `mvn clean test` em cada marco.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Teste unitário	Provar uma regra por API pública.	Depender de ordem aleatória não controlada.
blackjack.puml	Representar fatos atuais ou alvo claramente rotulado.	Misturar os dois sem aviso.
README	Ensinar executar e registrar decisões.	Virar cópia extensa da implementação.

Testes-alvo

Cenário	Evidência observável
Carta inválida	Tipo/validação impede construção.
Baralho completo	52 combinações únicas, 13 por naipe e 4 por rank.
Mãos independentes	Alterar uma não altera outra.
Distribuição	2 + 2 nas mãos e 48 restantes.
Ases	Todos os cenários da matriz passam.
Rodada	Transições, banca e resultados cobertos.
PUML	Renderiza e corresponde ao código.
README	Outra pessoa consegue compilar, testar, executar e explicar o escopo.

Armadilhas previsíveis

- Considerar cobertura percentual como prova de qualidade.
- Testar detalhes privados e tornar refatorações dolorosas.
- Exigir que shuffle sempre produza ordem diferente.
- Atualizar UML para um futuro ideal e chamá-lo de estado atual.
- Deixar regras importantes apenas em comentários.

Pronto quando

- ☐ Cada comportamento crítico possui cenário feliz, limite e erro.
- ☐ Uma falha aponta para uma regra específica.
- ☐ PUML renderiza e suas relações existem no código.
- ☐ README declara escopo e variante de regras.
- ☐ O build limpo passa antes de cada marco.

Explique em voz alta

- Que bug cada teste impediria?
- O teste falharia pelo motivo correto?
- O PUML descreve o código de hoje ou a intenção de amanhã?
- Uma pessoa nova descobriria a regra de soft 17 sem ler o corpo dos métodos?

CAPÍTULO 04

04

Matriz completa de testes

Os testes abaixo são a especificação executável do MVP. Implemente-os por fase, não todos de uma vez.

Unidade	Tema	Evidência	Fase
CartaTest	conserva rank e naipe	Carta válida mantém a identidade recebida.	F1/F2
CartaTest	rejeita dados inválidos	Null e rank impossível não criam objeto.	F2
CartaTest	texto e igualdade	Representação e política de objeto de valor são coerentes.	F2
BaralhoTest	composição completa	52 únicas, 13 por naipe e 4 por rank.	F3
BaralhoTest	comprar	Remove uma e devolve a mesma referência.	F3
BaralhoTest	esgotamento	Não repete; vazio segue contrato.	F3
BaralhoTest	embaralhar	Preserva tamanho e conjunto.	F3
MaoTest	estado inicial	Começa vazia e conserva cartas adicionadas.	F1
MaoTest	independência	Duas mãos não compartilham coleção.	F1
MaoTest	figuras	J, Q e K contam 10.	F5
MaoTest	múltiplos ases	Todos os ajustes produzem o melhor total.	F5
MaoTest	natural e estouro	Distingue estados derivados.	F5
JogadorTest	nome	Conserva nome válido e rejeita política inválida.	F1
ParticipantesTest	mãos próprias	Jogador e banca não compartilham Mao.	F1
BlackjackTest	construção	Não cria dependências ocultas.	F4
BlackjackTest	distribuição	Duas cartas para cada e 48 restantes.	F4
BlackjackTest	turno humano	Pedir e parar respeitam estado.	F6
BlackjackTest	turno da banca	Política de 17 é obedecida.	F6
BlackjackTest	resultado	Vitória, derrota, empate, natural e estouros.	F6
Aceitação manual	interface	Entrada inválida, nome completo, rodada e saída.	F7

Modelo Dado / Quando / Então

FICHA DE TESTE

Dado que:
[estado inicial relevante]

Quando:
[uma única ação pública]

Então:
[resultado observável]

Qual erro este teste impediria?
[uma frase concreta]

Arrange, Act, Assert

Parte	Pergunta	Sinal de problema
Arrange	Que objetos e estado inicial o cenário exige?	Preparação maior que o comportamento testado.
Act	Qual única mensagem dispara o comportamento?	Várias ações importantes no mesmo teste.
Assert	Qual efeito público prova a regra?	Inspeção de campo privado ou muitas causas de falha.

Aleatoriedade sem testes frágeis

- Não afirme que embaralhar necessariamente muda a ordem.
- Verifique que o tamanho e o conjunto de cartas foram preservados.
- Para uma distribuição previsível, use um baralho preparado ou uma fonte de Random controlada.
- Evite repetir um teste aleatório até passar; isso esconde defeitos.

TESTE DE QUALIDADE

Um teste bom falha por uma regra compreensível. Se ele pode falhar apenas porque o sorteio foi raro, ele não é uma especificação confiável.

Testes que parecem bons, mas não bastam

Teste superficial	O que deixa escapar	Como fortalecer
Baralho tem 52	Não detecta 52 cartas iguais.	Verificar combinações e distribuição.
Pontuação A + K	Não detecta erro com múltiplos ases.	Cobrir A + A e A + A + 9.
Jogador tem Mao	Pode ser a mesma instância do Dealer.	Usar assertNotSame e alterar uma.
Shuffle mudou ordem	Pode falhar legitimamente.	Testar preservação ou injetar aleatoriedade.
App imprime resultado	Pode ocultar regra duplicada na interface.	Testar resultado no domínio.

Ordem de execução recomendada

1. Rode somente a classe de teste da fase durante o ciclo curto.
2. Rode toda a suíte antes de considerar a fase pronta.
3. Rode `mvn clean test` no marco para eliminar resíduos de compilação.
4. Se um teste antigo falhar, descubra se a regra mudou ou se houve regressão.
5. Nunca altere a expectativa apenas para fazer o teste passar sem justificar a nova regra.

CAPÍTULO 05

05

Laboratório de UML

O diagrama deve responder quem conhece quem, quais referências são conservadas e onde vive cada responsabilidade.

Problemas atuais de blackjack.puml

- Faltam `@startuml` e `@enduml`.
- Carta ainda mostra `criarNaipe`, método já removido.
- Baralho mostra chamadas como membros e um método `gerarBaralho` inexistente.
- Blackjack mostra `playBlackjack`, que pertence a App.
- Naipe não aparece.
- Jogador aparece possuindo `Mao`, mas o código atual ainda perde a referência local.
- `Mao` aparece contendo cartas, mas ainda não possui campo.
- As visibilidades não refletem os campos `package-private` atuais.

DUAS FOTOGRAFIAS

Enquanto o projeto muda, mantenha um diagrama 'como está' e outro 'modelo-alvo'. Misturá-los em uma única figura transforma intenções futuras em afirmações falsas sobre o código atual.

Modelo-alvo conceitual

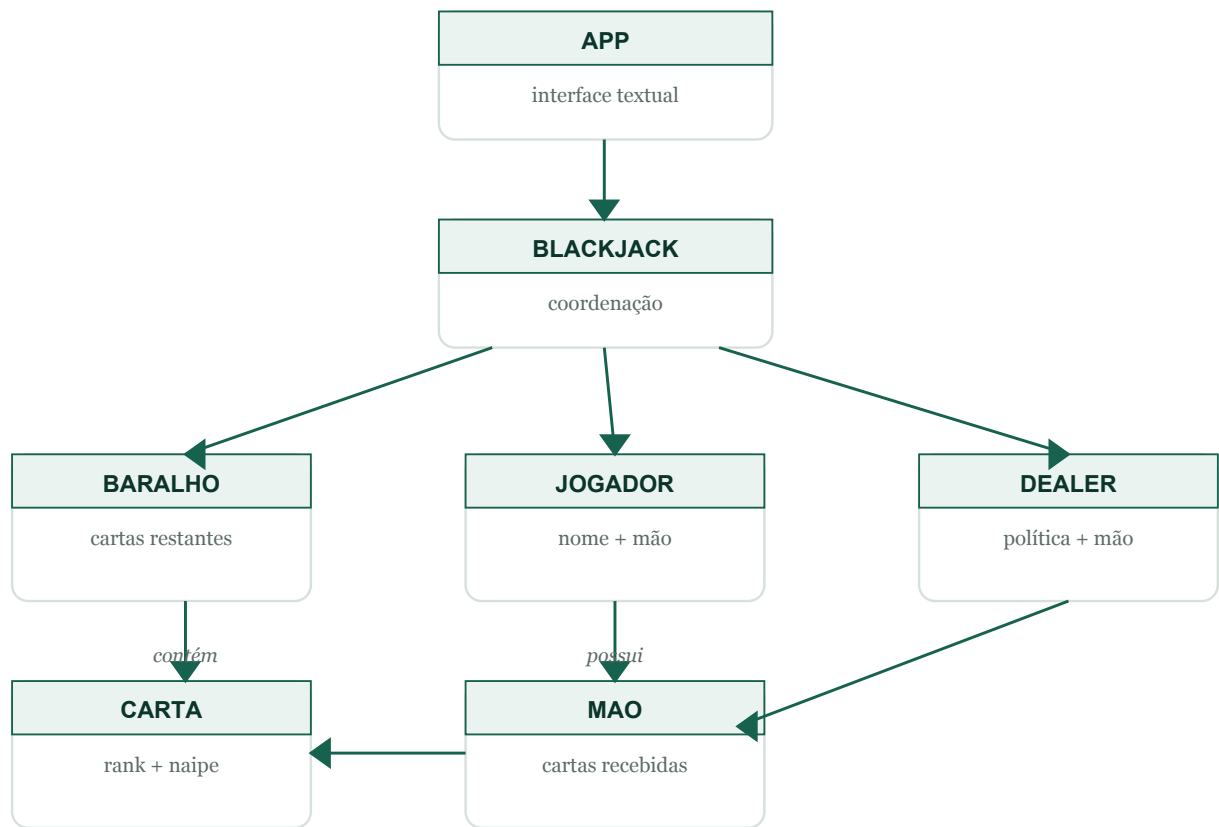


Figura 5. Colaborações principais do MVP, sem herança prematura.

Relações e multiplicidades

Relação	Leitura	Cuidado
App ..> Blackjack	App depende do jogo para iniciar e conduzir.	Não implica campo permanente.
Blackjack --> Baralho	O jogo conserva e usa um baralho.	A seta expressa navegabilidade.
Jogador *-- Mao	No MVP sem split, a mão pertence ao jogador.	Com split, a multiplicidade mudaria.
Dealer *-- Mao	A banca conserva sua mão.	Composição não é apenas 'tem um'.
Baralho --> Carta	Conhece cartas restantes.	Carta sai dessa coleção ao ser comprada.
Mao --> Carta	Conhece cartas recebidas.	Evite sugerir que a mesma carta está simultaneamente em ambos.
Carta --> Naipe	Carta conserva exatamente um naipe.	Enum também pode aparecer como tipo de atributo.

Esqueleto PlantUML para você completar

PLANTUML

```
@startuml
title Blackjack - modelo [ATUAL ou ALVO]

class Carta {
    - valor: ValorCarta
    - naipe: Naipe
    + [consultas públicas]
}

' Declare as outras classes com membros reais.
' Adicione relações somente quando o código conservar referências.

@enduml
```

REGRA UML

Uma relação no diagrama deve poder ser apontada no código: qual campo ou parâmetro estabelece o conhecimento? Se não houver resposta, talvez seja apenas uma dependência temporária ou uma intenção futura.

Checklist de sincronização do diagrama

- ☐ O arquivo possui delimitadores PlantUML.
- ☐ Todas as classes e enums atuais aparecem.
- ☐ Atributos têm nome, tipo e visibilidade.
- ☐ Métodos têm nome, parâmetros e retorno.
- ☐ Métodos removidos desapareceram do diagrama.
- ☐ Métodos não foram colocados na classe errada.
- ☐ Relações correspondem a referências reais.
- ☐ Multiplicidades refletem o escopo atual do MVP.
- ☐ Composição é usada somente quando a afirmação de posse é intencional.
- ☐ O título declara se o diagrama é atual ou alvo.
- ☐ O arquivo renderiza sem erro.

Perguntas de revisão

- Qual lado do relacionamento conserva a referência?
- A seta descreve uma dependência temporária ou uma associação persistente?
- O diamante está no lado do todo?
- Split mudaria a multiplicidade entre Jogador e Mao?
- Há duplicação suficiente para justificar uma classe Participante agora?

CAPÍTULO 06

06

Contratos do modelo-alvo

Uma assinatura é uma promessa. Esta matriz orienta o desenho sem entregar os corpos dos métodos.

Classe	Operação	Pré-condição	Pós-condição
Carta	construção	Rank e Naipe válidos.	Objeto imutável e consultável.
Carta	valor-base	Carta válida.	Número base; ás continua contextual.
Mao	adicionar	Carta não nula.	Quantidade aumenta em um.
Mao	calcularPontuacao	Coleção válida.	Melhor total com ajustes de ases.
Mao	estourou	Nenhuma.	total > 21.
Mao	blackjackNatural	Nenhuma.	21 com exatamente duas cartas.
Baralho	embaralhar	Baralho existente.	Mesmas cartas, outra permutação possível.
Baralho	comprar	Contrato de vazio definido.	Remove e devolve uma carta.
Jogador	construção	Nome válido.	Nome e mão própria conservados.
Dealer	deveComprar	Mão pontuável.	Decisão conforme política de 17.
Blackjack	construção	Dependências não nulas.	Conserva sem criar substitutos.
Blackjack	distribuirInicial	Baralho suficiente; estado adequado.	2 cartas por participante.
Blackjack	pedirCartaJogador	Turno humano ativo.	Transfere uma carta ou finaliza por estouro.
Blackjack	executarDealer	Turno da banca.	Compra até política mandar parar.
Blackjack	resultado	Rodada finalizada.	Enum de resultado.
App	ler opção	Linha disponível.	Escolha válida ou nova tentativa.

COMO USAR A TABELA

Antes de escrever um método, copie apenas sua linha para o caderno. Escreva um teste que observe a pós-condição. Só então escolha a estrutura interna.

Invariantes por classe

Classe	Invariante
Carta	Rank e naipe sempre válidos; identidade não muda.
Mao	Coleção existe, não contém null e representa todas as cartas recebidas.
Baralho	Coleção representa apenas cartas restantes; compra não duplica.
Jogador	Nome válido e uma mão própria no MVP.
Dealer	Uma mão própria e política de compra conhecida.
Blackjack	Dependências válidas; estado permite apenas operações coerentes.
App	Entrada inválida é tratada; regras não são duplicadas na interface.

Sinais de encapsulamento saudável

- Campos são privados por padrão.
- Operações expressam verbos do domínio: comprar, adicionar, calcular, parar.
- Coleções internas não são devolvidas para mutação externa.
- Consultas derivam estado em vez de duplicá-lo em booleans.
- Construtores impedem objetos parcialmente válidos.
- A interface não sabe como a pontuação é calculada.

ENCAPSULAMENTO NÃO É

Colocar private em tudo e gerar getters/setters automaticamente. Encapsular é proteger invariantes por meio de uma API que só permite ações válidas.

CAPÍTULO 07

07

Definição de pronto do MVP

Use estas páginas como checklist de aceitação. Não marque por intenção; marque somente com evidência.

Domínio e estado

- ☐ Carta não aceita rank ou naipe inválido.
- ☐ Carta é imutável e possui representação legível.
- ☐ Baralho novo contém 52 combinações únicas.
- ☐ Cada mão conserva sua própria coleção.
- ☐ Jogador e Dealer não compartilham Mao.
- ☐ Uma carta comprada deixa de estar no Baralho.
- ☐ A distribuição inicial deixa 48 cartas restantes.
- ☐ Pontuação funciona com figuras e múltiplos ases.
- ☐ Blackjack natural é diferente de 21 com três cartas.
- ☐ Estado da rodada impede ações fora de hora.

Fluxo da partida

- ☐ Jogador consegue pedir ou parar.
- ☐ Estouro do jogador encerra a rodada imediatamente.
- ☐ Banca compra abaixo de 17.
- ☐ Banca para em 17, inclusive soft 17 no MVP.
- ☐ Vitória, derrota, empate, natural e estouros são reconhecidos.
- ☐ Nenhuma compra acontece depois de FINALIZADA.
- ☐ O usuário consegue iniciar nova rodada ou sair.

Qualidade técnica

- ☐ mvn clean test termina com BUILD SUCCESS.
- ☐ Os testes estão separados por responsabilidade.
- ☐ Cenários aleatórios são determinísticos quando necessário.
- ☐ Campos de estado são privados e referências obrigatórias são finais.
- ☐ Não há listas internas mutáveis expostas.
- ☐ Não há objetos construídos apenas para serem substituídos.
- ☐ Não há imports inutilizados importantes ou métodos vazios.
- ☐ Não há regra importante implementada somente no App.
- ☐ O código usa uma língua e uma convenção de nomes consistentes.

Documentação e modelo

- ☐ README declara escopo e regras adotadas.
- ☐ README mostra compilar, testar e executar.
- ☐ PUML possui delimitadores e renderiza.
- ☐ PUML contém Naípe e ValorCarta.
- ☐ Atributos, operações e visibilidades correspondem ao código.
- ☐ Relações e multiplicidades correspondem às referências reais.
- ☐ Extensões futuras estão separadas do MVP.

EVIDÊNCIA MÍNIMA

Cada marca precisa apontar para pelo menos uma destas provas: teste automatizado, execução manual reproduzível, código revisado ou diagrama renderizado.

Roteiro de aceitação manual

1. Inicie a aplicação e confirme que o menu mostra sair.
2. Digite texto onde se espera número e confirme nova tentativa.
3. Informe um nome com espaços.
4. Inicie uma rodada e observe duas cartas para cada participante.
5. Confirme que uma carta da banca permanece oculta até o momento decidido.
6. Peça cartas até parar ou estourar.
7. Observe o turno automático da banca.
8. Confira total e resultado com as cartas exibidas.
9. Escolha jogar novamente e confirme estado novo.
10. Escolha sair e confirme encerramento sem exceção.

Registro de evidência

Passo	Resultado esperado	Observado	Correção necessária
1	Inicie a aplicação e confirme que o menu mostra sair.		
2	Digite texto onde se espera número e confirme nova tentativa.		
3	Informe um nome com espaços.		
4	Inicie uma rodada e observe duas cartas para cada participante.		
5	Confirme que uma carta da banca permanece oculta até o momento decidido.		
6	Peça cartas até parar ou estourar.		
7	Observe o turno automático da banca.		
8	Confira total e resultado com as cartas exibidas.		
9	Escolha jogar novamente e confirme estado novo.		
10	Escolha sair e confirme encerramento sem exceção.		

CAPÍTULO 08

08

Plano de estudo em doze sessões

Uma cadência curta transforma o roadmap em prática deliberada e reduz mudanças grandes demais.

Sessão	Foco	Fase	Saída observável
1	Baseline e referências	F0	Grafo de objetos explicado; build limpo.
2	Mao persistente	F1	Mão vazia e adicionar carta testados.
3	Participantes	F1	Mãos próprias e nome válido.
4	ValorCarta	F2	Nenhuma carta inválida nasce.
5	Baralho completo	F3	52 combinações comprovadas.
6	Comprar e embaralhar	F3	Transferência e vazio testados.
7	Blackjack limpo	F4	Sem dependências temporárias.
8	Distribuição inicial	F4	2 + 2 e 48 restantes.
9	Pontuação	F5	Matriz de ases passando.
10	Estados e banca	F6	Turnos e política de 17.
11	Interface	F7	Rodada manual completa.
12	UML, README e aceitação	F8	Checklist final e build limpo.

Ritual de cada sessão

1. Preveja o comportamento atual.
2. Rode o teste mais próximo da mudança.
3. Escreva ou ajuste um cenário que falhe pelo motivo esperado.
4. Faça a menor alteração útil.
5. Rode a classe de teste e depois toda a suíte.
6. Explique a responsabilidade em uma frase.
7. Registre no PUML ou README se uma decisão mudou.

LIMITE SAUDÁVEL

Se uma sessão altera mais de duas responsabilidades, pare e divida. Mudanças pequenas tornam o erro localizável e preservam o aprendizado.

Ficha reutilizável de implementação

Pergunta	Minha resposta
Problema observado	
Classe responsável	
Dados que essa classe já possui	
Contrato do método	
Pré-condição	
Pós-condição	
Teste que falha antes	
Menor implementação possível	
Armadilha que quero evitar	
Evidência de conclusão	

EXPLIQUE ANTES DE CODIFICAR

Se você não consegue completar 'esta operação pertence a X porque X já possui...', o lugar do método ainda não está claro.

CAPÍTULO 09

09

Pistas graduais

Consulte somente depois de tentar. Cada nível revela um pouco mais, sem fornecer classes completas.

1. Mao não conserva cartas

PISTA 1 - PERGUNTA

Que diferença existe entre uma variável declarada dentro e fora do construtor?

PISTA 2 - CONCEITO

O estado que precisa sobreviver deve ser campo de instância.

PISTA 3 - DIREÇÃO

Procure uma declaração `private final List<Carta>` no corpo da classe e inicialização no construtor.

2. Jogador perde a mão

PISTA 1 - PERGUNTA

Quem aponta para a Mao depois que o construtor termina?

PISTA 2 - CONCEITO

A referência local desaparece; o participante precisa conservá-la.

PISTA 3 - DIREÇÃO

O campo e o parâmetro podem ter o mesmo tipo; use `this` para selecionar a instância.

3. Carta aceita XPTO

PISTA 1 - PERGUNTA

Qual tipo permite apenas treze possibilidades?

PISTA 2 - CONCEITO

Um enum fecha o domínio e move erros para compilação.

PISTA 3 - DIREÇÃO

Faça Carta receber ValorCarta e faça Baralho percorrer os valores oficiais.

4. Comprar não reduz o baralho

PISTA 1 - PERGUNTA

O método devolve uma carta ou remove uma carta?

PISTA 2 - CONCEITO

Transferir exige remover da coleção de origem.

PISTA 3 - DIREÇÃO

Escolha o topo e use uma operação de remoção que também devolva o elemento.

5. Distribuição cria cartas novas

PISTA 1 - PERGUNTA

De onde deveria vir cada carta distribuída?

PISTA 2 - CONCEITO

Blackjack coordena a mesma referência entre Baralho e Mao.

PISTA 3 - DIREÇÃO

A sequência conceitual é comprar e depois adicionar; não use `new Carta` na distribuição.

6. A + A soma 22

PISTA 1 - PERGUNTA

Quantos ases foram contados inicialmente como 11?

PISTA 2 - CONCEITO

Ajuste um ás por vez enquanto o total exceder 21.

PISTA 3 - DIREÇÃO

Cada ajuste subtrai exatamente 10; repita enquanto ainda houver ás flexível.

7. Teste de shuffle falha às vezes

PISTA 1 - PERGUNTA

A mesma ordem é matematicamente impossível?

PISTA 2 - CONCEITO

Aleatoriedade não promete mudança, apenas uma distribuição.

PISTA 3 - DIREÇÃO

Teste preservação do conjunto ou forneça Random previsível.

8. Nome completo quebra o menu

PISTA 1 - PERGUNTA

O que `next ( )` deixa no buffer?

PISTA 2 - CONCEITO

Ele lê um token, não a linha inteira.

PISTA 3 - DIREÇÃO

Leia linha completa e converta opções textuais de maneira controlada.

CAPÍTULO 10

10

Glossário aplicado

Definições curtas ligadas ao projeto, com a confusão que cada termo ajuda a evitar.

Termo	Definição aplicada ao Blackjack
Classe	Tipo que define estado e comportamento. Exemplo: Mao. Não é a mão concreta de um jogador.
Objeto	Instância criada em execução. Duas chamadas a new Mao produzem objetos distintos.
Instância	Sinônimo prático de objeto pertencente a uma classe.
Referência	Valor usado para alcançar um objeto. Atribuir uma referência não clona a Mao.
Campo	Variável declarada no corpo da classe e parte do estado de cada instância.
Variável local	Nome temporário declarado dentro de método ou construtor; sai do escopo ao terminar.
Parâmetro	Variável local que recebe o valor fornecido na chamada.
Escopo	Região do código em que um nome pode ser usado.
Alcançabilidade	Existência de um caminho de referências até um objeto; orienta a coleta de lixo.
Estado	Conjunto de dados que um objeto conserva entre chamadas.
Identidade	O fato de ser aquele objeto específico; testável com assertSame.
Igualdade lógica	Regra pela qual dois objetos representam o mesmo valor; usa equals.
Construtor	Rotina que estabelece o estado inicial válido de uma nova instância.
Invariante	Condição que deve permanecer verdadeira durante toda a vida válida do objeto.
Encapsulamento	Proteção de invariantes por uma API controlada; não é apenas usar private.
Visibilidade	Quem pode acessar um membro: private, package-private, protected ou public.
this	Referência para a instância atual. Em this.nome = nome, separa campo e parâmetro.
final	Impede reatribuir o campo após inicialização; não congela automaticamente o objeto referenciado.
static	Membro pertencente à classe e compartilhado; não possui this.
Método de instância	Operação executada sobre um objeto específico, como baralho.comprar().
Coleção	Objeto que agrupa referências, como uma List de cartas.
List	Interface de coleção ordenada que aceita duplicatas e, por padrão, null.
ArrayList	Implementação indexada de List; adequada ao baralho deste exercício.
Generics	Parâmetro de tipo que restringe List; não garante regras como unicidade.

Termo	Definição aplicada ao Blackjack
Enum	Tipo com conjunto fechado de instâncias nomeadas, adequado a Naipes e ValorCarta.
Imutabilidade	Estado observável não muda depois da construção; Carta é forte candidata.
Valor derivado	Dado calculado a partir de outro estado, como a pontuação da Mão.
Associação	Relação persistente de conhecimento entre objetos.
Dependência	Uso temporário; App depende de Blackjack mesmo sem guardá-lo como campo.
Agregação	Relação UML fraca de todo-parte; muitas vezes associação simples é mais clara.
Composição	Posse forte e ciclo de vida associado; no MVP, participante e Mão.
Herança	Relação 'é um'. Não deve surgir apenas para eliminar um campo repetido.
Coesão	Grau em que uma classe reúne responsabilidades relacionadas.
Acoplamento	Quantidade e força das dependências entre componentes.
Responsabilidade	Motivo principal para uma classe existir e mudar.
Contrato	Promessa de pré-condições, efeitos, retorno e erros de uma operação.
API pública	Conjunto de operações pelas quais outros objetos colaboram sem acessar detalhes.
Injeção de dependência	Receber colaboradores prontos, como Blackjack recebe Baralho e participantes.
Exceção	Sinalização de que uma operação não pode cumprir seu contrato.
Teste unitário	Verificação automatizada de uma pequena regra por comportamento observável.
Arrange-Act-Assert	Organização do teste em preparar, agir e verificar.
Determinismo	Mesma preparação produz mesmo resultado, essencial para testes confiáveis.
Máquina de estados	Modelo de fases e transições permitidas da rodada.
Garbage collector	Mecanismo que pode recuperar objetos inalcançáveis; não age em instante previsível.
Pass-by-value	Java copia sempre um valor; para objetos, copia o valor da referência.
Aliasing	Duas referências alcançam o mesmo objeto ou coleção.
Cópia defensiva	Nova coleção criada para impedir que mudanças externas alterem o estado interno.
Mão macia	Mão em que ao menos um ás ainda conta como 11.
Blackjack natural	Total 21 formado por exatamente duas cartas.

ENCERRAMENTO

Sua próxima ação concreta

COMECE POR MAO

Transforme a coleção local em estado de instância, ofereça adicionar e quantidade, e prove com testes que duas mãos são independentes. Não implemente pontuação ainda.

Sequência das próximas três entregas

1. Mao conserva cartas e seus testes passam.
2. Jogador e Dealer conservam mãos próprias e seus testes passam.
3. Carta recebe um tipo fechado de valor nominal e o Baralho continua com 52 combinações válidas.

Perguntas finais de professor

- Quem possui os dados necessários para executar esta regra?
- O valor precisa sobreviver ao método ou é apenas temporário?
- Que estado inválido o construtor deve impedir?
- Qual comportamento público prova que a mudança funcionou?
- O PUMML descreve o que existe hoje?
- Eu consigo explicar a decisão sem apontar apenas para a sintaxe?

Um bom projeto orientado a objetos não nasce de muitas classes. Ele nasce quando cada objeto conserva o estado pelo qual é responsável, protege suas invariantes e colabora por mensagens claras.

Fim do roteiro - agora a próxima linha é sua.